

Domain 5

Context Management & Reliability

15% of exam · 5 task statements

5.1 Context windows & prompt caching · 5.2 Long-context strategies · 5.3 Error propagation · 5.4 Human-in-the-loop escalation · 5.5 Self-evaluation patterns

5.1 Context Windows & Prompt Caching

Token budgeting and stable-prefix caching

The context window — fundamentals

Every Claude API call has a context window holding both input AND output tokens. Context includes: system prompt + conversation history + tool definitions + tool results + current message. Managing what enters this window is the core skill of Domain 5.

Concept	What to know
~0.75 words per token	Rough estimate — exact counts vary by content type
Input + output share the window	Reserve space for the response — don't fill the window with input alone
Tool definitions consume tokens	Every tool schema in your API call uses context space
count_tokens endpoint	Pre-flight check before expensive long-context calls — never guess

Prompt caching — how it works

Prompt caching marks stable, repeated sections with `cache_control` so Claude reuses the processed KV cache on subsequent calls. Only the changed suffix (e.g. the user's question) is processed fresh.

What to cache (stable content)

- System prompt / instructions
- Large reference documents sent every call
- Tool definitions that don't change
- Few-shot examples
- Content identical across many API calls

Don't cache (variable content)

- User messages (change every turn)
- Tool results (unique per call)
- Dynamic context injected per request
- Content that changes frequently

cache_control placement — the rule

```
// Place cache_control at the END of content you want cached
{
  "role": "system",
  "content": [{
    "type": "text",
    "text": "You are a support agent...\n\n[500-line policy document]",
    "cache_control": { "type": "ephemeral" } // marks end of cached prefix
  }]
}
```

```
}  
  
// Everything BEFORE the marker is cached.  
  
// The user message that follows is processed fresh each time.  
  
// Cache hits reduce cost by up to 90% and cut TTFT significantly.
```

KEY CONCEPT

Exam pattern: a system sending the same large system prompt + policy document on every interaction is the ideal caching scenario. Cache the stable prefix — only the user's message is billed at full price on cache hits.

EXAM TRAP — PROMPT CACHING

'Prompt caching is only useful for very long contexts.' Wrong — it's valuable whenever the same stable content repeats across calls. The pattern: stable prefix + variable suffix = cache the prefix, regardless of total length.

5.2 Long-Context Strategies

Lost in the middle, progressive summarisation, and RAG

The 'lost in the middle' effect

Information placed in the MIDDLE of a long input receives less attention than content at the beginning or end. This is documented LLM behaviour — not a hallucination, but a reliability risk for long inputs.

Anti-pattern (loses context)

- Key findings buried in the middle of a 200K token context
- Critical instructions sandwiched between documents
- Important context in the middle of a long message history

Correct pattern

- Place key summaries and findings at the BEGINNING of long inputs
- Put critical instructions in the system prompt (always at the start)
- Use a persistent 'case facts' block at the top of every prompt

Progressive summarisation — and when NOT to use it

Progressive summarisation compresses older turns into a running summary to keep context manageable. It works for general conversation — but is DANGEROUS for transactional data.

Safe to summarise

- General conversation turns
- Background context and preferences
- Explanations and reasoning chains
- Content where paraphrasing doesn't lose meaning

NEVER summarise

- Order numbers, invoice IDs, ticket IDs
- Monetary amounts (£47.23 becomes 'around £50')
- Dates and deadlines
- Account numbers and reference codes
- Any data that must be reproduced exactly

KEY CONCEPT

The correct pattern: extract all transactional specifics into a persistent 'case facts' block at the top of every prompt. This block is NEVER summarised — always included verbatim. Only general conversation context around it gets summarised.

Case facts block — structure

```
// Persistent case facts block — always verbatim, never summarised  
  
Customer ID: CUST-847291  
Order ID: ORD-2024-11-47823  
Order date: 2024-11-14  
Items: 2x Widget Pro (SKU: WP-001) @ £47.23 each  
Refund requested: £94.46  
Refund reason: Items not received
```

Account status: Gold member since 2021

Customer contacted support on 2024-11-21. Confirmed delivery address.

Agent checked tracking – no delivery scan since Nov 15.

[... summarised context ...]

RAG — when to retrieve rather than include everything

Strategy	When to use	Trade-off
Full context inclusion	Small documents (<50K tokens total)	Simple but expensive/slow at scale
RAG (semantic search)	Large knowledge bases, many documents	Relevant retrieval, but retrieval quality matters
Prompt caching + RAG	Repeated queries against same large doc set	Best of both — cache stable content, retrieve variable

EXAM TRAP — SUMMARISATION ON TRANSACTIONS

A support agent uses progressive summarisation. After many turns, it processes a refund for the wrong amount — '£94.46' was summarised to 'approximately £90.' Fix: persistent case facts block, always verbatim.

5.3 Error Handling & Propagation Graceful degradation and structured error messages

The 3 error categories in multi-agent systems

Category	What it means	Example
Tool errors	External system failure — the tool call itself failed	API timeout, permission denied, invalid MCP response
Reasoning errors	Claude made a wrong inference or decision	Misidentified document type, called the wrong tool
Environment errors	Infrastructure-level failure outside Claude's control	Network outage, rate limit hit, context window exceeded

Graceful degradation — the core principle

When one component fails, do NOT terminate the entire workflow. Continue with what succeeded, annotate what failed, and surface the gap transparently.

Hard termination (wrong)

- One subagent fails → entire pipeline stops
- User receives nothing
- No information about what succeeded
- Silent failure with no context

Graceful degradation (correct)

- One subagent fails → others continue
- Partial results returned with gap annotation
- 'Section 3 could not be retrieved — access error'
- User gets maximum value from what succeeded

KEY CONCEPT

Error propagation rule: when propagating to the coordinator, always include: (1) failure type, (2) what was attempted, (3) partial results obtained, (4) alternative approaches available. Never return a generic failure.

What to include in an error propagation message

```
// Complete error propagation – gives coordinator everything needed
{
  "status": "partial_failure",
  "failure_type": "permission_error",
  "tool_attempted": "fetch_financial_records",
```

```
"what_was_attempted": "Retrieve Q3 revenue data from finance system",
"partial_results": {"Q1": {...}, "Q2": {...}}, // Q3 failed, Q1+Q2 succeeded
"alternatives": ["request_manual_export", "use_summary_report_tool"],
"is_retryable": false
}
```

Fallback strategies by error type

Error type	First action	If first action fails
Transient tool error	Retry with exponential backoff (up to 3×)	Return partial results + annotation to coordinator
Permission error	Return structured error immediately (not retryable)	Escalate with alternative data source suggestions
Context window exceeded	Truncate or summarise non-critical content	Split into smaller sub-tasks, process incrementally
Reasoning error detected	Self-evaluate output against criteria	Re-attempt with clarified constraints, then escalate

EXAM TRAP — ERROR PROPAGATION

'One subagent in a 4-subagent pipeline hits a permission error — terminate the whole pipeline.' Wrong. Terminate the failing subagent, return partial results from others, annotate the gap, let the coordinator decide.

5.4 Human-in-the-Loop Escalation

Valid triggers, structured handoffs, and policy gaps

The 3 valid escalation triggers — memorise exactly

VALID TRIGGERS

1. Customer explicitly requests a human — always honour, no exceptions
2. Policy gap exists — agent's instructions don't cover this situation
3. Unable to make meaningful progress — repeated attempts failed, continuing wastes time

The 2 invalid escalation triggers — exam traps both

EXAM TRAP — INVALID TRIGGER 1 — CUSTOMER SENTIMENT

Escalating because the customer seems frustrated or upset. Sentiment is NOT a valid escalation trigger. A frustrated customer whose problem CAN be solved should have it solved — not handed off. Escalate on policy gaps and task impossibility, not emotions.

EXAM TRAP — INVALID TRIGGER 2 — MODEL SELF-CONFIDENCE

Escalating because Claude's internal confidence score is low, or because Claude 'isn't sure.' Self-reported confidence is NOT reliable — Claude can be confidently wrong and uncertain about correct answers. Use programmatic criteria and explicit policy rules — not Claude's self-assessment.

KEY CONCEPT

The principle: escalation criteria must be programmatic and objective. Both sentiment and self-confidence produce inconsistent escalation decisions. The exam always rewards programmatic enforcement of explicit escalation rules.

Structured handoff — complete package

Human agents don't have the full conversation transcript. The handoff must be self-contained:

```
// Complete handoff package — human agent needs all of this
{
  "customer_id": "CUST-847291",
  "customer_name": "Jane Smith",
  "issue_summary": "Order ORD-2024-11-47823 not received",
}
```

```

"root_cause": "No delivery scan since Nov 15 – likely lost in transit",
"actions_attempted": ["checked tracking", "contacted courier API – timeout"],
"recommended_action": "Process full refund of £94.46 and file courier claim",
"account_tier": "Gold member",
"escalation_reason": "Courier API unavailable – cannot confirm delivery status"
}

```

Complex but in-policy vs policy gap

Complex but in-policy → handle it	Policy gap → escalate immediately
- Agent has authority to proceed - However complex, existing rules cover it - Example: large refund within approved limit - Example: multi-item return within policy	- No instructions cover this situation - Agent does NOT improvise - Example: refund request outside return window - Example: service not in the product catalogue
EXAM TRAP — ESCALATION — COMPLEXITY	'The request is complex, so escalate to a human.' Wrong. Complexity alone is not an escalation trigger. Escalate when policy is SILENT — not when the case is merely difficult but within policy.

5.5 Self-Evaluation & Production Reliability

Independent review, monitoring, and quality gates

Why same-session self-review is weak

The same reasoning that produced the error will rationalise it during review. Claude confirms its own logic rather than challenging it. Self-review has a systematic bias toward approving its own output — it catches surface errors but misses logical ones.

Independent review with a separate Claude instance

Same-session self-review (weak)	Independent review (correct)
- Same Claude instance reviews its own output - Same reasoning context = same biases - Tends to approve own output - Catches surface errors, misses logical ones	- Separate Claude instance with fresh context - Reviewer system prompt with explicit criteria - No access to generator's reasoning - Catches errors the generator rationalised away
KEY CONCEPT	Architecture: Generator instance → output → Reviewer instance (fresh context, reviewer prompt, explicit criteria) → pass/fail verdict + specific issues → if fail, targeted retry → Generator.

When to use each review approach

Use case	Pattern	Why
Checking basic format compliance	Self-evaluation acceptable	Format errors are objective — same session can catch them
Verifying logical correctness	Independent review required	Same reasoning context misses logical errors
Critical data accuracy (financial, medical)	Independent review + human review	High-stakes — both AI and human checkpoints needed
Detecting reasoning errors in agentic output	Independent review required	Generator can't see its own blind spots

Production monitoring — closing the reliability loop

Signal to monitor	What it tells you
Validation failure rate	Rising rate → prompt drift or data distribution shift
Retry loop frequency	High retries → schema or criteria mismatch
Escalation rate	Sudden spike → new edge case type hitting policy gap
Tool selection accuracy	Misrouting increase → tool description drift or new similar tool added
EXAM TRAP — SELF-REVIEW BIAS	'Add a second prompt asking Claude to review its own answer in the same conversation.' Same-session self-review — same reasoning context approves its own errors. Fix: SEPARATE Claude instance with fresh context and explicit reviewer criteria.
EXAM TRAP — CONFIDENCE AS QUALITY	Using Claude's self-reported confidence as a quality gate. Claude's internal confidence is not calibrated — it can be confidently wrong. Use independent review with explicit criteria.

All Domain 5 Exam Traps		Memorise before exam day
EXAM TRAP — PROMPT CACHING	'Prompt caching is only useful for very long contexts.' Wrong — it's valuable whenever the same stable content repeats across calls. Pattern: stable prefix + variable suffix = cache the prefix.	
EXAM TRAP — PROGRESSIVE SUMMARISATION	Using progressive summarisation on transactional data. Amounts, IDs, and dates get paraphrased. Fix: persistent case facts block, always verbatim, never summarised.	
EXAM TRAP — LOST IN THE MIDDLE	Placing key findings or instructions in the middle of a long context. They receive less attention. Fix: key content at the BEGINNING of long inputs.	
EXAM TRAP — ERROR PROPAGATION	One subagent fails → terminate the entire pipeline. Wrong. Continue with what succeeded, annotate the gap, return partial results.	
EXAM TRAP — ERROR PROPAGATION	Generic 'subagent failed' error to the coordinator. Wrong — coordinator needs: failure type + what was attempted + partial results + alternative approaches.	
EXAM TRAP — ESCALATION — SENTIMENT	Escalating because customer seems frustrated. Sentiment is NOT a valid trigger. Escalate on policy gaps and task impossibility only.	
EXAM TRAP — ESCALATION — CONFIDENCE	Escalating because Claude's self-reported confidence is low. Confidence is not calibrated. Use programmatic criteria.	
EXAM TRAP — ESCALATION — COMPLEXITY	'The request is complex, so escalate.' Wrong. Complexity alone is not a trigger. Escalate when policy is SILENT.	
EXAM TRAP — SELF-REVIEW	'Ask Claude to review its own answer in the same conversation.' Same-session bias. Fix: SEPARATE instance, fresh context, explicit reviewer criteria.	
EXAM TRAP — CONFIDENCE GATE	Using Claude's self-confidence score as a quality gate. Not calibrated to actual accuracy. Use independent review with explicit criteria.	

ALL 5 DOMAINS COMPLETE

Domain 1: Agentic Architecture (27%) · Domain 2: Tool Design & MCP (18%) · Domain 3: Claude Code (20%) · Domain 4: Prompt Engineering (20%) · Domain 5: Context & Reliability (15%)

Passing score: 720 / 1000 · 60 scenario-based questions · 120 minutes · 4 scenarios drawn at random from 6